

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills		
Student Name:	Juhi Surin	Reg. No.:	192525148
Date:	05-08-2026	Duration:	60 minutes

Code of Conduct

I **Juhi Surin (Reg. No. 192525148)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Juhi Surin

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an individual software project problem statement. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.

Component Interaction and Quality Attribute Analysis	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis	/5
Selection of Software Architecture	/5
Architecture Diagram and Component Design	/5
Component Interaction and Quality Attribute Analysis	/5
Architectural Justification and Technical Presentation	/5
TOTAL	/5

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ARCHITECTURE DESIGN REPORT

CO2 Assessment Tool 2 – Software Engineering (CSA10)

Assigned Problem Statement (Q37):

“AI-Based Digital Waste Recycling Marketplace and Resource Recovery System”

Submitted by:	Juhi Surin
Registration Number:	192525148
Course:	CSA10 – Software Engineering
Faculty In-Charge:	Latha R
Department:	Computer Science and Engineering
Date of Submission:	05 August 2026

1. Introduction and Problem Statement

1.1 Industry Context

Rapid urbanization has significantly increased municipal and industrial waste generation across cities. Governments and environmental organizations are increasingly promoting circular economy practices to improve recycling rates and resource recovery. Emerging technologies such as Artificial Intelligence (AI), the Internet of Things (IoT), cloud computing, and digital marketplace platforms make it possible to coordinate waste collection, segregation, trading, and recycling far more efficiently while reducing the overall environmental footprint.

1.2 Problem Statement

Existing waste management systems generally lack effective coordination between waste generators, recycling facilities, and buyers of recyclable materials, which results in poor recycling rates and increased dependency on landfills. The proposed system is an AI-powered digital marketplace that connects households, industries, recyclers, and waste collection agencies while automatically classifying recyclable materials using AI. The platform is expected to optimize waste collection schedules, recommend nearby recycling centres, facilitate the trading of recyclable materials, and generate sustainability analytics to strengthen resource recovery.

1.3 Objectives

- Digitize recyclable waste trading between generators and recyclers.
- Classify waste materials automatically using AI-based image recognition.
- Connect waste generators (households and industries) with recyclers.
- Optimize waste collection schedules for collection agencies.
- Monitor ongoing recycling activities across the platform.
- Generate sustainability analytics for stakeholders and regulators.
- Promote circular economy practices among all participants.
- Reduce landfill waste through better resource recovery.

1.4 Expected Outcomes

- Increased recycling rates across households, industries, and recyclers.
- Reduced environmental pollution from unmanaged waste.
- Improved efficiency of waste collection operations.
- Better resource recovery and material reuse.
- Enhanced collaboration among all stakeholders in the waste value chain.
- Reduced dependency on landfill disposal.
- A more sustainable, data-driven approach to waste management.
- Increased public participation in recycling initiatives.

2. System Requirements Analysis

2.1 System Objectives

The core objective of the system is to digitally connect every stakeholder in the recycling value chain – households, industrial waste generators, recyclers, and municipal collection agencies – through a single AI-driven marketplace that

automates waste classification, matches supply with demand for recyclable materials, and continuously reports on sustainability performance.

2.2 Functional Requirements

- User registration and role-based access for households, industries, recyclers, and collection agencies.
- AI-based image classification of waste into categories (plastic, metal, paper, e-waste, organic, etc.).
- Digital marketplace to list, browse, bid on, and trade recyclable materials.
- Matching engine that connects waste generators with the nearest suitable recycler.
- Collection scheduling module that plans and optimizes pickup routes and time slots.
- Recycling centre locator with map-based search and directions.
- IoT integration for smart bins that report fill-level, weight, and location.
- Payment and transaction processing for traded materials.
- Sustainability analytics dashboard for waste volumes, recycling rates, and carbon-offset estimates.
- Notification and alert system for pickups, trade confirmations, and schedule changes.

2.3 Non-Functional Requirements

- Scalability – the platform must support a growing number of users, sensors, and transactions without redesign.
- Performance – AI classification and matching results should be returned within a few seconds.
- Security – user data, payment details, and transaction records must be protected against unauthorized access.
- Reliability – the system must consistently process waste-classification and trading requests without data loss.
- Availability – critical services (marketplace, scheduling) must remain accessible with minimal downtime.
- Maintainability – individual features (e.g., analytics, AI model) should be upgradable without affecting the whole system.
- Usability – the mobile/web interface must be simple enough for households with limited technical knowledge.
- Interoperability – the system must integrate with external municipal and government reporting systems.

2.4 Key Quality Attributes Influencing the Architecture

- Scalability – to handle increasing numbers of users, IoT devices, and marketplace transactions.
- Maintainability – to allow independent updates to AI models, marketplace logic, and analytics.
- Reliability – to ensure accurate classification and trustworthy transaction processing.
- Availability – to keep the marketplace and scheduling services running with minimal interruption.
- Performance – to deliver near real-time AI classification and search/matching results.
- Security – to safeguard personal, location, and financial data of all stakeholders.

3. Selection of a Suitable Software Architecture

3.1 Selected Architecture

A Hybrid Architecture combining a Microservices Architecture for the core business capabilities with an Event-Driven Architecture (publish-subscribe messaging) for real-time IoT and AI data flow is selected for this system. RESTful APIs exposed through an API Gateway provide the client-facing Service-Oriented interface, while an internal event bus decouples the AI classification, IoT sensor, and analytics pipelines from the request-driven services.

3.2 Justification

- The problem statement requires several independent capabilities – user management, AI classification, marketplace/trading, scheduling, locator, and analytics – that have very different scaling and update needs, which Microservices support far better than a single monolithic application.
- IoT smart bins and AI classification generate continuous, asynchronous streams of data; an Event-Driven backbone (message/event bus) allows these streams to be processed without blocking user-facing requests.
- A digital marketplace naturally involves multiple external and internal parties (households, industries, recyclers, agencies, payment gateways, government portals), which aligns well with a Service-Oriented style of exposing well-defined APIs.
- Microservices allow the compute-intensive AI/ML classification engine to be scaled independently of lighter services such as notifications or the locator service, optimizing cloud cost and performance.
- The hybrid style supports incremental rollout – new services (e.g., a carbon-credit module) can be added later without redesigning the existing system.

3.3 Architectures Considered and Rejected

- Monolithic Layered Architecture – rejected because a single deployable unit would make it difficult to scale the AI engine independently and would slow down releases as the system grows.
- Pure Client-Server Architecture – rejected because it does not natively support the asynchronous, high-volume IoT sensor and event data this system must handle.
- Pure Serverless Architecture – considered for individual functions (e.g., notification dispatch) but not selected as the primary style, since the always-on AI inference workload and stateful marketplace logic are more predictable and cost-effective on managed microservices.

4. Software Architecture Diagram

The diagram below illustrates the complete structure of the proposed AI-Based Digital Waste Recycling Marketplace and Resource Recovery System, including the client layer, API gateway, microservices, the AI classification engine, the event/message bus, the data layer, and external system integrations.

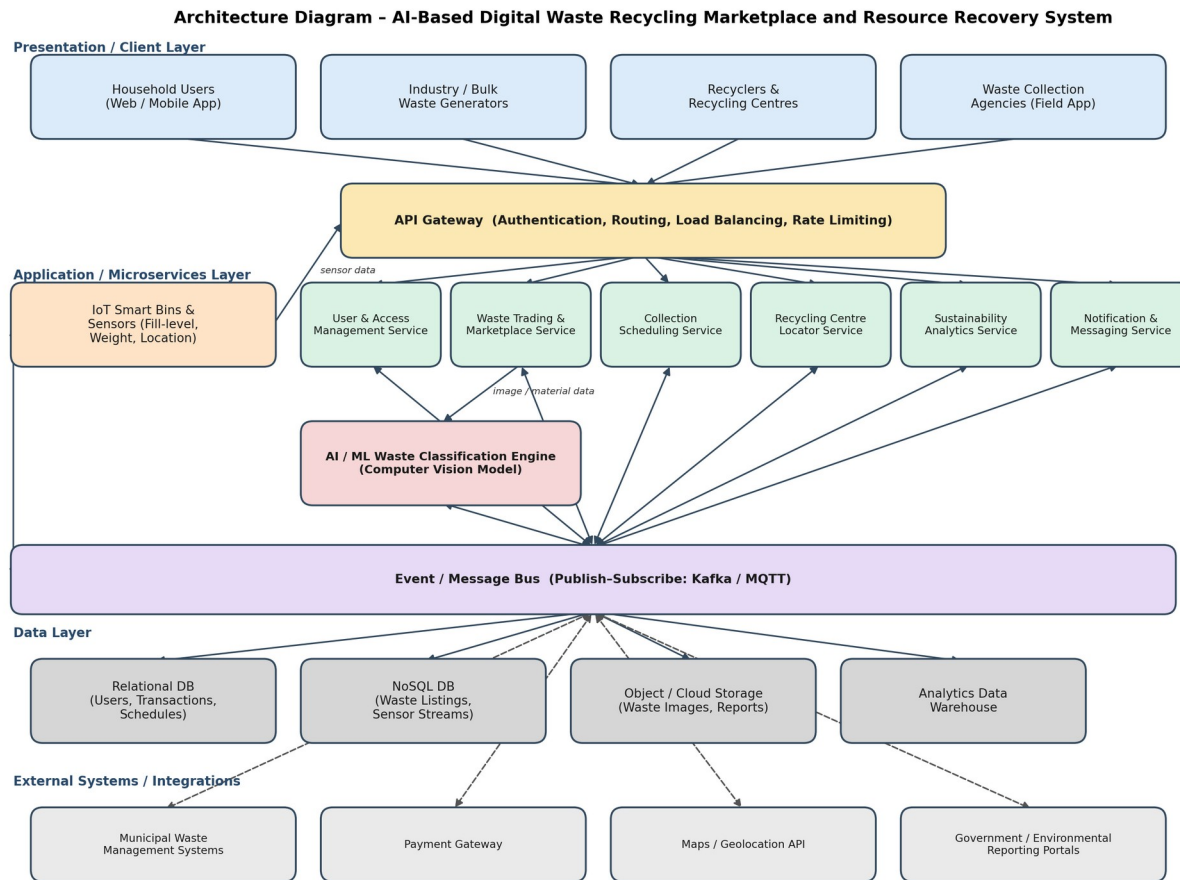


Figure 1: Architecture diagram of the AI-Based Digital Waste Recycling Marketplace and Resource Recovery System.

4.1 Notation and Layer Summary

- **Presentation / Client Layer** – web and mobile applications used by households, industries, recyclers, and field collection staff.
- **API Gateway** – single entry point handling authentication, request routing, load balancing, and rate limiting.
- **Application / Microservices Layer** – independently deployable services for user management, marketplace/trading, scheduling, locator, analytics, and notifications.
- **AI/ML Engine** – a dedicated computer-vision service that classifies uploaded waste images into material categories.
- **Event/Message Bus** – a publish–subscribe backbone (e.g., Kafka/MQTT) that carries IoT sensor data and inter-service events asynchronously.
- **Data Layer** – polyglot persistence with a relational database, a NoSQL store, cloud/object storage, and an analytics data warehouse.
- **External Systems** – municipal waste-management systems, payment gateways, maps/geolocation APIs, and government reporting portals.

5. Components and Their Interactions

5.1 Component Responsibilities

Component	Responsibility
API Gateway	Authenticates requests, routes traffic to the correct microservice, applies load balancing and rate limiting.
User & Access Management Service	Handles registration, login, roles (household, industry, recycler, agency), and profile data.
Waste Trading & Marketplace Service	Manages listings of recyclable materials, bidding/trading logic, and transaction records.
AI/ML Waste Classification Engine	Analyzes uploaded waste images using a computer-vision model and returns the material category and confidence score.
Collection Scheduling Service	Plans and optimizes pickup schedules and routes for waste collection agencies.
Recycling Centre Locator Service	Recommends the nearest suitable recycling centre using geolocation data.
Sustainability Analytics Service	Aggregates platform data to generate recycling-rate, landfill-reduction, and sustainability reports.
Notification & Messaging Service	Sends alerts for pickups, trade confirmations, and schedule changes to all stakeholders.
IoT Smart Bins & Sensors	Continuously report fill-level, weight, and location data to support proactive collection.
Event/Message Bus	Decouples services by carrying asynchronous events and sensor streams between producers and consumers.
Data Layer (SQL/NoSQL/Storage/Warehouse)	Persists transactional data, sensor streams, waste images, and analytical datasets.

5.2 Communication Between Components

Client applications communicate with backend services exclusively through the API Gateway using REST/HTTPS calls, which keeps the internal service topology hidden from external users. Synchronous, request-response interactions (e.g., a household requesting nearby recycling centres) are handled directly through the gateway to the relevant microservice. Asynchronous interactions – such as IoT sensor readings, AI classification results, and cross-service notifications – are published to the event/message bus, where interested services subscribe to the relevant topics. This publish-subscribe pattern decouples producers from consumers, so a slow or unavailable analytics service, for example, cannot block the marketplace or scheduling services.

5.3 Overall System Workflow

- A household or industry user uploads a photo of the waste item through the mobile/web app.
- The request passes through the API Gateway to the Waste Trading & Marketplace Service, which forwards the image to the AI/ML Waste Classification Engine.

- The AI engine classifies the material type and publishes the result back to the Marketplace Service, which automatically creates a categorized listing.
- The Marketplace Service matches the listing with a suitable recycler based on material type, location, and demand.
- Once a trade is confirmed, the Collection Scheduling Service optimizes a pickup slot and route for the Waste Collection Agency.
- IoT smart bins continuously publish fill-level and location data to the event bus, allowing the scheduling service to proactively plan collections even without a manual listing.
- The Recycling Centre Locator Service assists recyclers and agencies in identifying the nearest processing facility.
- The Notification Service informs all relevant parties (pickup confirmed, trade completed, schedule changed) in real time.
- The Sustainability Analytics Service continuously consumes events from the bus to update recycling-rate dashboards and generate periodic sustainability reports for regulators and platform administrators.

6. Discussion of Quality Attributes

Quality Attribute	How the Architecture Addresses It
Scalability	Each microservice (e.g., AI engine, marketplace) can be scaled independently and horizontally behind the API Gateway; the event bus absorbs bursts of IoT and transaction traffic without overloading any single service.
Security	The API Gateway centralizes authentication, authorization, and rate limiting; role-based access control restricts data visibility per stakeholder; sensitive data (payments, personal details) is encrypted in transit and at rest.
Performance	Compute-intensive AI classification runs as a dedicated, independently scaled service; asynchronous event processing keeps user-facing requests fast by offloading background work (e.g., analytics aggregation) to the event bus.
Reliability	Services are loosely coupled through the event bus, so a failure in one service (e.g., analytics) does not directly crash another; message queues provide retry and replay capability for failed operations.
Maintainability	Independent microservices can be updated, redeployed, or replaced (e.g., upgrading the AI model) without redesigning or redeploying the entire platform, reducing regression risk.
Availability	Stateless services behind the gateway can run multiple replicas across availability zones; the event-driven design allows temporarily unavailable services to catch up once restored, rather than causing data loss.

7. Architectural Justification and Trade-offs

7.1 Rationale Behind the Architectural Choices

The hybrid Microservices plus Event-Driven architecture was chosen because the system genuinely combines two different workload patterns: request-driven marketplace transactions (best served by independently scalable microservices behind an API Gateway) and continuous, high-volume sensor/AI data streams (best served by an asynchronous event bus). Choosing either style alone would have compromised one side of the system – a pure

microservices/REST design would strain under constant IoT streaming, while a purely event-driven design would complicate simple, synchronous marketplace operations such as browsing listings.

7.2 Trade-offs Considered

- Complexity vs. Flexibility – microservices introduce operational complexity (service discovery, monitoring, deployment pipelines) compared to a monolith, but this is accepted in exchange for independent scalability and easier long-term maintenance.
- Consistency vs. Availability – the event-driven design favours eventual consistency (e.g., analytics dashboards updating a few seconds after a trade) to keep core services highly available; this is acceptable since analytics do not require instant consistency.
- Latency vs. Decoupling – routing all cross-service communication through an event bus adds a small amount of latency compared to direct calls, but this is a reasonable trade-off for the reliability and loose coupling gained.
- Cost vs. Performance – running a dedicated, GPU-backed AI classification service increases infrastructure cost but is necessary to keep classification response times acceptable for end users.

7.3 Support for Future Enhancements and Long-Term Maintainability

- New services (for example, a carbon-credit marketplace or a reward/gamification module) can be added as independent microservices that simply subscribe to existing events, without modifying current services.
- The AI/ML classification model can be retrained or replaced independently as more waste-image data becomes available, without any downtime for the rest of the platform.
- Integration with additional external systems, such as new government reporting portals or third-party logistics providers, can be added at the API Gateway and event-bus level without disrupting existing stakeholders.
- The polyglot data layer allows each service to choose the most suitable storage technology as requirements evolve, supporting long-term scalability and maintainability.
- Because services are independently deployable, the development team can adopt CI/CD pipelines to release updates frequently and with lower risk, supporting continuous improvement of the platform.

8. Conclusion

The AI-Based Digital Waste Recycling Marketplace and Resource Recovery System requires an architecture that can support real-time AI classification, high-volume IoT sensor data, and a multi-stakeholder digital marketplace at the same time. A hybrid Microservices and Event-Driven Architecture, exposed through a Service-Oriented API Gateway, satisfies these needs by allowing each capability to scale, evolve, and fail independently while remaining loosely coupled through an asynchronous event bus. This architecture directly supports the system's core quality attributes – scalability, security, performance, reliability, maintainability, and availability – and provides a solid, extensible foundation for future enhancements toward a more circular and sustainable waste-management ecosystem.